ADAM Consensus Quorum Resiliency and Placeholder Security Analysis

1. Executive Summary

This document evaluates the security posture and cryptographic safety of the quorum-resilient block template generation and validation mechanism in the **ADAM (A Decentralized Approach Model)** consensus framework of the KRISTA network.

Previously, the cooperative mining loop required a 100% response rate from all elected nodes (11-14 miners in Fallback Mode, 11 miners in Standard Mode) to successfully construct and propagate a block. Under this model, if even a single node went offline, delayed its solution, or experienced a network partition, the Coordinator could not build a block template. This resulted in chain freezes and compromised network liveness.

To resolve this, we introduced a **Quorum-Resilient Responding and Placeholder Mechanism**:

1. The block template is finalized and propagated as long as a minimum quorum of valid solutions is met (`GetAdamThreshold(nHeight)`):
   - **Bootstrap Phase (`nHeight < nAdamThresholdBootstrapLimit` , i.e., below block 50,000 on Mainnet / 100,000 on Testnet / 1,000 on Regtest)**: At least **5** valid solutions from the elected miners.
   - **Standard Phase (`nHeight >= nAdamThresholdBootstrapLimit` , Version 11 & Version 12)**: At least **7** (`nAdamThreshold` on Mainnet/Regtest) or **3** (on Testnet) valid solutions from the elected miners.
2. Missing solutions are represented within the block header's serialization format using **empty vector placeholders** (`std::vector<unsigned char>()`).
3. This analysis demonstrates that the placeholder mechanism preserves the cryptographic security of the consensus model, maintains backward compatibility, and mitigates key attack vectors (including forgery, coordinator censorship, payout theft, tampering, and replay attacks) while significantly improving network liveness.

---

2. Technical Mechanism and Validation Workflow

2.1. Placeholder Insertion during Block Template Generation

When the Coordinator's staking/mining thread (`CreateNewBlock()`) attempts to construct a block template:

1. It queries the active elected miners list for the block's seed (`SelectAdamNodes()`).
2. It fetches available partial PoW puzzle solutions matching these elected miners' public keys from the network's consensus cache.
3. If the number of collected valid solutions is less than the required quorum threshold (T), the block template generation is deferred.
4. If the number of valid solutions meets or exceeds T, but some elected miners are missing, the Coordinator substitutes the missing miner solutions with an empty byte vector:

vAdamSolutions[i] = std::vector < unsigned char > ()

5. This ensures that `vAdamSolutions` preserves a 1:1 positional mapping with the elected miners listed in `vAdamMiners`.

2.2. Network Validation Logic

Upon receiving a block, every validating peer executes the consensus verification checks in `CheckBlock()` and `ContextualCheckBlockHeader()`:

1. **Array Boundary Validation**: Check that the size of `vAdamSolutions` matches the expected number of miners (e.g., N in Version 12).
2. **Quorum Verification**:
   - Loop through `vAdamSolutions`.
   - If a solution is empty (`std::vector<unsigned char>()`), it is treated as a placeholder. The verification function `VerifyAdamSolution()` immediately returns `false` (or is bypassed), and this placeholder is **not** counted toward the valid solutions.
   - If a solution is non-empty, the peer validates the partial PoW target hash difficulty and verifies the miner's cryptographic signature against the elected miner's public key. If both checks pass, the valid solutions count is incremented.
3. **Threshold Enforcement**: The block is rejected with a consensus violation error if the count of cryptographically validated, non-empty solutions is less than the required threshold (T).

---

3. Threat Modeling & Security Mitigations

3.1. Cryptographic Forgery & Bypass Protection

- **Threat**: An attacker attempts to use empty placeholders to bypass Proof-of-Work checks or forge miner signatures, submitting a block with fewer than the required number of physical solutions.
- **Mitigation**:
  - Empty placeholders are mathematically incapable of satisfying signature or difficulty checks. The validation code explicitly treats them as failed solutions.
  - The block verification rules enforce that at least T = GetAdamThreshold($nHeight$) (5 during bootstrap, 7 on Mainnet/Regtest post-bootstrap, or 3 on Testnet post-bootstrap) solutions must be *fully valid, non-empty, cryptographically signed, and meet the target difficulty*.
  - An attacker cannot bypass the physical work requirement; they must still perform the necessary multi-algorithm hashing computations for at least T seats to build a block that the network will accept.

3.2. Coordinator Abuse & Miner Censorship

- **Threat**: A malicious Coordinator deliberately censors honest miners by omitting their solutions and replacing them with empty placeholders, attempting to shut them out of block participation.
- **Mitigation**:
  - The degree of censorship a Coordinator can perform is strictly capped by the quorum threshold T = GetAdamThreshold($nHeight$).

- In Fallback Mode ($N \in \{11, \ldots, 14\}$) and Standard Mode ($N = 11$), with $T = 5$ during bootstrap, $T = 7$ on Mainnet/Regtest post-bootstrap, and $T = 3$ on Testnet post-bootstrap, the Coordinator can censor at most $N - T$ miners (e.g., 4 on Mainnet/Regtest post-bootstrap when $N = 11$).
- If a Coordinator attempts to censor more miners, the block will fail validation at all peer nodes and be rejected.
- In addition, because the Masternode network dynamically rotates coordinators and miners every block height using a Verifiable Random Function (VRF) powered by deterministic hybrid signatures, a malicious coordinator only has a temporary opportunity to censor. They cannot lock out a miner indefinitely.

3.3. Block Reward and Payout Security (No Financial Incentive to Censor)

- **Threat**: A Coordinator excludes a miner's solution to steal their portion of the block reward or redirect it to their own address or remaining validator seats.
- **Mitigation**:
  - The block reward distribution is governed deterministically by consensus rules at the protocol level. Payout addresses are calculated by querying `SelectAdamNodes()` for the elected public keys at that block height.
  - Under Model D (`IsModelDActive`), **Equal Work, Equal Reward** applies across all 11 seats: every seat $i \in \{0, \ldots, 10\}$ that provides a valid PoW solution in `vAdamSolutions[i]` — whether filled by the original primary validator `vExpectedMiners[i]` OR by a legitimate same-track standby validator `vAdamMiners[i]` when the primary seat timed out ($t \geq 60$s) — receives its full track-specific share of the 40% Participant Validator reward pool (20% across the 7 Masternode Validator quota seats at `~2.857%` each + 20% across the 4 Independent Miner quota seats [`8.00%` for 1 Coin-Lock + `4.00%` each for 3 Open PoW]). Unsolved/empty seats receive `0%`, while the Coordinator retains its full `15%` reward (`nProducerSplit`) when rescuing a stalled block with standby validators. Any unclaimed empty-seat shares (`nUnclaimedPartSplit = (nPartSplitTotal - nPaidPartTotal)`) are split **50% to the Automatic Protocol Burn (`nExpectedBurn = nUnclaimedPartSplit / 2`)** and **50% to the Developer Treasury (`nExpectedTreasury = nTreasurySplit + (nUnclaimedPartSplit - nExpectedBurn)`)** while the Passive Masternode Winner receives its exact `35%` (`nMNSplit`).
  - Furthermore, in `qblockprop` (evaluated only when the LLMQ member's tip matches `msg.block.hashPrevBlock`), at $t \geq 60$s LLMQ quorum members verify their local `mapAdamSolutionsCache[hashPrevBlock]` and refuse to sign (`QUORUMSIGSHARE`) any block proposal that substitutes a primary seat (`vAdamMiners[i] != vExpectedMiners[i]`) when a valid solution from `vExpectedMiners[i]` is already cached locally. Because neither the Coordinator's net reward (`0` net gain), nor the Passive Masternode Winner's 35% share, nor the fixed reward of any active seat increases when a seat is omitted or substituted, a Coordinator has **zero financial incentive** to censor honest validators.

3.4. Replay and Tampering Attacks

- **Threat**: An attacker replays valid solutions from a previous block or tries to modify transaction data in the block template after the miners have signed their solutions.
- **Mitigation**:
  - **Seed Binding**: Each miner signs a unique puzzle hash derived from the rolling election seed:

    $$\text{PuzzleHash} = \text{CalculateAdamPuzzleHash}(\text{algoIndex}, \text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i)$$

The rolling seed `Seed_H` is derived from the previous block's VRF proof and is unique to the current block height. A solution signed for height H cannot be replayed at height $H + 1$ because the seeds will not match, causing signature validation to fail.

- **Header Integrity**: The final block header hash binds all transactions (via `hashMerkleRoot`), the block time, the previous block hash, the VRF proof, and the list of elected miners.

- **Dual Signatures**: After the block template is finalized (with or without placeholders), the Coordinator signs the entire block header hash (`vAdamCoordinatorSig`) using the hybrid BLS12-381 + ECDSA fallback signature mechanism (`SignBLSWithECDSAFallback` / `VerifyBLSWithECDSAFallback`), where a BLS signature is authorized by an ECDSA signature of the corresponding BLS public key. At heights ≥ 200 (Cooperative PoS), the staker also signs the block using their staking key (`vchBlockSig`). Any alteration of transactions or block metadata invalidates these overarching signatures, preventing any post-hoc tampering by intermediate nodes.

3.5. Nothing-at-Stake and Sybil Resiliency

- **Threat**: Attackers sign blocks on multiple forks at zero cost or spin up virtual nodes to dominate the leader election.
- **Mitigation**:
  - The requirement for physical PoW computation on T elected nodes ensures that creating blocks on competing forks remains computationally expensive. Staking on multiple forks is not free, mitigating nothing-at-stake.
  - Under the **Dual-Track Quota Election (`ElectDualTrackValidators`, active at `nAdamDualTrackHeight = UPGRADE_ADAM`)**, the 11 validator seats are partitioned into **Track 1 (8 Collateralized Seats: 7 Active Masternodes [`4,200 KRISTA`] + 1 Coin-Lock [`250 KRISTA`] with intra-Track-1 spillover)** and **Track 2 (3 Open PoW Seats: `10 KRISTA` `PoW-Lock` / `Zero-Coin PoW`)**, while the Coordinator is elected from active Masternodes whenever active Masternodes exceed the threshold. Because `ADAM_MASTERNODE_TRACK_QUOTA = 7` is locked at compile time (`static_assert`) and verified at runtime in `chainparams.cpp` (`assert(ADAM_MASTERNODE_TRACK_QUOTA >= consensus.nAdamThreshold)`), Active Masternodes alone always satisfy the maximum post-bootstrap threshold (`GetAdamThreshold = 7`) even if all 4 non-Masternode seats withhold solutions. Furthermore, if any Masternode is offline while non-Masternode seats withhold solutions, **Standby Validator Seat Handover (`ADAM_COMMITTEE_FALLBACK_TIMEOUT_SECS = 60s`)** hands silent seats only to same-track standby validators in strict sequential rank order (`MN -> MN`, `Coin-Lock -> Coin-Lock`, `Open PoW -> Open PoW`, verified against the block's `CCoinsViewCache* pView`, with equal per-seat track reward [`~2.857%` MN / `8.00%` Coin-Lock / `4.00%` Open PoW] for any solved seat, full 15% rescue Coordinator reward, LLMQ anti-censorship protection for primary solutions in `qblockprop`, and unclaimed empty-seat shares split 50% to the Automatic Protocol Burn and 50% to the Developer Treasury), and **Time-Relaxed Quorum Threshold (`ADAM_THRESHOLD_RELAXATION_TIMEOUT_SECS = 120s`)** progressively lowers the threshold by 1 per 60s slot down to `ADAM_MIN_RELAXED_THRESHOLD = 5` (`> 4` non-MN seats, ensuring at least 1 Active Masternode is always required) whenever post-bootstrap OR active Masternodes ≥ 7 (and `3` during bootstrap only when active Masternodes < 7), preventing any permanent block stall.

---

4. Conclusion

The introduction of the quorum-resilient placeholder mechanism represents a major security and robustness upgrade for the KRISTA network. It successfully:

- **Restores Liveness**: Eliminates the single point of failure where a single offline or lagging miner could freeze the entire blockchain.
- **Preserves Safety**: Enforces the same cryptographic and thermodynamic security guarantees of the ADAM consensus model by ensuring that a substantial majority of elected validators must actively participate and sign.
- **Zero Incentive for Abuse**: Eliminates any financial incentive for coordinator censorship by decoupling the physical solution inclusion from the deterministic reward payout logic.